ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

ĐỀ TÀI

TRỰC QUAN HÓA THUẬT TOÁN

Học phần: Phân tích và thiết kế thuật toán
Giảng viên: Huỳnh Thị Thanh Thương

Lớp: CS112.F31.LT.TTNT
Sinh viên: Trần Phú Đạt
MSSV: 26410018

Năm học: 2026–2027

Mục lục

1. Giới thiệu và động cơ lựa chọn 3
2. Phát biểu và mô hình bài toán 4
3. Thiết kế cấu trúc dữ liệu và thuật toán.....5
4. Ví dụ minh họa từng bước.....8
5. Thiết kế giao diện và quá trình phát triển.....15
6. Thiết kế thực nghiệm và bộ testcase 18
7. Kết quả kiểm thử và đánh giá.....20
8. Hạn chế và hướng phát triển.....21
9. Tài liệu tham khảo 22

Phụ lục A. Cách chạy và cấu trúc gói nộp 23

Phụ lục B. Nhật ký sử dụng AI.....24

1. Giới thiệu và động cơ lựa chọn

1.1. Bối cảnh

Đồ thị là mô hình dữ liệu dùng để biểu diễn các đối tượng và mối quan hệ giữa chúng. Một đỉnh đại diện cho một đối tượng; một cạnh biểu diễn khả năng kết nối hoặc quan hệ giữa hai đối tượng. Khi cạnh có trọng số, trọng số có thể mô tả khoảng cách, chi phí, thời gian hoặc mức tiêu hao tài nguyên. Những đặc điểm này khiến đồ thị phù hợp để mô hình hóa mạng máy tính, hệ thống giao thông, quan hệ giữa các thực thể và bài toán lập kế hoạch đường đi.

Bốn thuật toán trong GraphLab cùng làm việc trên đồ thị nhưng thể hiện bốn chiến lược đáng so sánh: BFS mở rộng theo lớp bằng hàng đợi; DFS đi sâu bằng ngăn xếp; Dijkstra chọn đỉnh có khoảng cách tạm thời nhỏ nhất; A* kết hợp chi phí đã đi với ước lượng chi phí còn lại. Nếu chỉ nhìn kết quả cuối cùng, người học dễ bỏ qua lý do một đỉnh được chọn, cách parent thay đổi hoặc vì sao một tuyến đường đang tốt hơn tuyến đã biết.

1.2. Lý do lựa chọn

Đề tài được chọn vì cân bằng giữa độ khó triển khai, giá trị học thuật và khả năng trình bày bằng giao diện trực quan. Cùng một mô hình đồ thị có thể được dùng cho nhiều thuật toán, giúp quan sát sự khác biệt của cấu trúc dữ liệu và chiến lược chọn đỉnh mà không phải thay đổi toàn bộ bài toán. Đây cũng là cơ hội kết hợp kiến thức phân tích độ phức tạp, cấu trúc dữ liệu, thuật toán tìm kiếm và phát triển giao diện web.

Điểm nhấn của đề tài không phải chỉ đưa ra một đường đi, mà làm cho tiến trình giải có thể quan sát và kiểm tra được: người dùng có thể tiến/lùi từng bước, xem frontier, visited, bảng khoảng cách, đỉnh/cạnh đang xét và kết quả cuối cùng. Để thể hiện mức đầu tư vượt mức tối thiểu, GraphLab bổ sung A*, chế độ chạy tự động, điều chỉnh đồ thị, đồ thị có hướng/vô hướng, kiểm tra dữ liệu không hợp lệ, bộ testcase tự động và một bài thuyết trình web tương tác.

Cụ thể giải quyết tìm kiếm/đường đi trên đồ thị có trọng số(khác với tìm đường đi ngắn nhất).

1.3. Mục tiêu và phạm vi

Các mục tiêu cụ thể:

1. Cung cấp một đồ thị mẫu và cho phép thay đổi nguồn, đích, hướng đồ thị, thêm đỉnh, thêm cạnh, xóa cạnh cuối và khôi phục dữ liệu mẫu.
2. Cài đặt BFS, DFS, Dijkstra và A* trên cùng mô hình dữ liệu.
3. Lưu lại trạng thái của mỗi thao tác để có thể xem tuần tự, lùi lại, reset và tự phát lại.
4. Hiển thị kết quả cuối gồm đường đi, chi phí phù hợp với từng thuật toán, số đỉnh đã thăm và số lần kiểm tra cạnh.
5. Kiểm thử các trường hợp thường, biên, không liên thông, đồ thị có hướng, trọng số 0/âm, dữ liệu sai và so sánh thuật toán đường đi ngắn nhất với một oracle độc lập.

Phạm vi hiện tại là đồ thị nhỏ phục vụ giảng dạy, giới hạn tối đa 15 đỉnh được thêm từ giao diện. Đây không phải phần mềm định tuyến sản xuất. Giao diện dùng vị trí đỉnh cố định/tự sinh; chưa có kéo-thả, import file hoặc editor tổng quát cho toàn bộ danh sách đỉnh/cạnh.

2. Phát biểu và mô hình bài toán

2.1. Phát biểu bài toán

Cho đồ thị có trọng số (G=(V,E)), đỉnh bắt đầu (s), đỉnh đích (t) và một thuật toán được chọn trong tập {BFS, DFS, Dijkstra, A*}. Hệ thống cần thực thi thuật toán từ (s), trình bày trạng thái trung gian sau từng thao tác, sau đó trả về đường đi tìm được hoặc thông báo không có đường đi. Với Dijkstra và A*, mục tiêu là tìm đường đi có tổng trọng số nhỏ nhất. Với BFS, đường đi dựng từ cây khám phá có số cạnh nhỏ nhất; trọng số cạnh không được dùng để quyết định thứ tự BFS. Với DFS, hệ thống trả về đường đi theo cây khám phá nếu đến được đích; đường này không được bảo đảm tối ưu.

2.2. Input

- V: danh sách các đỉnh, mỗi đỉnh có mã định danh duy nhất và tọa độ hiển thị (x, y).
- E: danh sách các cạnh (from, to, weight); mã cạnh dùng để đánh dấu cạnh đang xét.
- directed: true nếu đồ thị có hướng, false nếu vô hướng.
- algorithm: BFS, DFS, Dijkstra hoặc A*.
- start, goal: mã đỉnh nguồn và đích.

Trong giao diện, người dùng thay đổi đồ thị bằng các điều khiển thêm đỉnh, thêm cạnh, xóa cạnh cuối và khôi phục đồ thị mẫu. Cạnh mới có trọng số số học hữu hạn. Giao diện không cho thêm cạnh tự nối đỉnh với chính nó. Dijkstra và A* từ chối trọng số âm; BFS/DFS không tối ưu theo trọng số và không dùng trọng số để chọn đường.

2.3. Output

Kết quả gồm hai nhóm:

Kết quả cuối: - path: danh sách đỉnh từ nguồn đến đích; rỗng nếu không tồn tại đường. - totalCost: tổng trọng số với Dijkstra/A*; số cạnh với BFS/DFS; null nếu không có đường. - visitedCount: số đỉnh được thăm trong quá trình chạy. - edgeChecks: số lần thuật toán kiểm tra cạnh. - Trạng thái hoàn tất, không có đường đi hoặc input không hợp lệ.

Execution trace: mỗi bước lưu tiêu đề thao tác, diễn giải, đỉnh đang xét, danh sách visited, frontier, bảng distances, quan hệ previous, cạnh đang xét, bộ đếm kiểm tra cạnh và trạng thái. Riêng A*, trace lưu thêm scores (giá trị (f=g+h)) và heuristics (giá trị (h)). Các snapshot là dữ liệu mà giao diện dùng để tái hiện lịch sử thuật toán.

2.4. Điều kiện và ràng buộc

1. Nguồn và đích phải nằm trong tập đỉnh; mã đỉnh không được trùng.
2. Mỗi cạnh phải tham chiếu đến đỉnh có tồn tại; trọng số phải là số hữu hạn.
3. Dijkstra và A* chỉ chấp nhận trọng số không âm.
4. Đồ thị có hướng chỉ cho phép di chuyển theo chiều cạnh; đồ thị vô hướng tạo lân cận ở cả hai đầu.
5. Nếu start == goal, kết quả là đường đi gồm đúng một đỉnh và chi phí 0.
6. Nếu đích không thể đến từ nguồn, đường đi rỗng và chi phí là null.
7. Trường hợp nhiều đường tối ưu cùng chi phí, đường cụ thể phụ thuộc thứ tự cạnh và quy tắc phá hòa; tiêu chí đúng là tính hợp lệ của đường và chi phí tối ưu.

2.5. Ví dụ Input/Output

Đồ thị vô hướng mẫu có các cạnh:

Cạnh	Trọng số	Cạnh	Trọng số	Cạnh	Trọng số
A–B	4	A–C	2	B–C	1
B–D	5	C–D	8	C–E	10
D–E	2	D–F	6	E–F	3

Nguồn A, đích F:

- BFS trả về đường ít cạnh theo thứ tự lân cận; với dữ liệu mẫu và thứ tự cạnh hiện tại, kết quả có 3 cạnh.
- DFS trả về đường theo nhánh được mở rộng trước; không khẳng định đây là đường ngắn nhất.
- Dijkstra trả về A → C → B → D → E → F, tổng trọng số 2 + 1 + 5 + 2 + 3 = 13.
- A* dùng heuristic nhất quán từ tọa độ hiển thị và trả về chi phí tối ưu 13 trên đồ thị mẫu.

3. Thiết kế cấu trúc dữ liệu và thuật toán

3.1. Mô hình dữ liệu

Mô hình TypeScript được định nghĩa trong src/graph.ts:

- GraphNode: { id, x, y } — định danh và vị trí hiển thị.
- GraphEdge: { id, from, to, weight } — cạnh và trọng số.
- GraphData: danh sách đỉnh, danh sách cạnh, cờ có hướng.
- Step: snapshot của trạng thái sau thao tác.
- RunResult: trace, đường đi, chi phí, số đỉnh đã thăm và số lần kiểm tra cạnh.

Thuật toán xây dựng danh sách kề Map<NodeId, Neighbor[]> một lần trong mỗi lần chạy. Danh sách kề phù hợp với đồ thị thưa và cho phép duyệt các cạnh đi ra nhanh hơn so với việc quét toàn bộ danh sách cạnh ở mỗi đỉnh. Với đồ thị vô hướng, mỗi cạnh được thêm vào danh sách kề của cả hai đầu; với đồ thị có hướng chỉ thêm theo chiều from → to.

distances[v] lưu số cạnh từ nguồn đối với BFS/DFS, hoặc chi phí đường tốt nhất đã biết g(v) đối với Dijkstra/A. *previous[v] lưu đỉnh cha để dựng lại đường đi. BFS dùng mảng làm hàng đợi FIFO; DFS dùng mảng như ngăn xếp LIFO; Dijkstra duy trì tập đỉnh chưa cố định và tìm khoảng cách nhỏ nhất bằng quét tuyến tính; A* duy trì tập open, closed và điểm ưu tiên f(v)=g(v)+h(v).

3.2. BFS — Breadth-First Search

BFS đánh dấu đỉnh đã phát hiện ngay khi thêm vào hàng đợi để tránh lặp. Mỗi vòng lấy phần tử đầu queue, kiểm tra đích, rồi xét các đỉnh kề. Các đỉnh mới được ghi previous[v]=u, distances[v]=distances[u]+1 và thêm vào cuối queue.

Bất biến: các đỉnh được lấy ra khỏi queue theo số cạnh không giảm tính từ nguồn. Vì vậy, nếu mọi cạnh được xem là có cùng chi phí, đường BFS tìm được tới một đỉnh có số cạnh tối thiểu.

```
BFS(G, s, t):
  queue ← [s]; seen ← {s}; previous[s] ← null; distance[s] ← 0
  while queue không rỗng:
    u ← pop_front(queue)
    ghi snapshot(u)
    nếu u = t: dừng
    với mỗi v kề u:
      nếu v chưa thuộc seen:
        seen.add(v); previous[v] ← u
        distance[v] ← distance[u] + 1
        push_back(queue, v)
  truy vết previous để trả về đường đi
```

Với danh sách kề, phần lõi BFS có độ phức tạp O(V+E) thời gian và O(V) bộ nhớ phụ trợ. Phiên bản GraphLab còn ghi snapshot sau thao tác; chi phí lưu lịch sử được phân tích riêng ở mục 3.6.

3.3. DFS — Depth-First Search

DFS dùng ngăn xếp LIFO. Khi lấy một đỉnh ra, hệ thống ghi nhận đã thăm rồi lần lượt thêm các đỉnh kề chưa phát hiện vào stack. Vì phần tử được thêm sau cùng được lấy trước, thứ tự cạnh ảnh hưởng đến nhánh DFS khám phá. previous được gán tại lần phát hiện đầu tiên.

Bất biến: mọi đỉnh trong stack đã được phát hiện từ nguồn nhưng chưa được mở rộng. DFS phù hợp minh họa cách khám phá sâu và quay lại các nhánh còn lại; đường đi tìm được không bảo đảm ít cạnh hoặc ít trọng số nhất.

Với danh sách kề, phần lõi DFS có độ phức tạp O(V+E) thời gian và O(V) bộ nhớ phụ trợ. Trong bản cài đặt, đỉnh được đánh dấu khi được đẩy vào stack nên không có nhiều bản sao chờ cho cùng một đỉnh.

3.4. Dijkstra

Dijkstra giải bài toán đường đi ngắn nhất từ nguồn đến các đỉnh khác khi mọi trọng số cạnh không âm. Thuật toán khởi tạo dist[s]=0, các khoảng cách còn lại là vô cùng. Mỗi vòng chọn đỉnh chưa cố định u có dist[u] nhỏ nhất; với mỗi cạnh (u,v,w), tính candidate=dist[u]+w. Nếu candidate < dist[v], cập nhật dist[v] và previous[v]=u.

Bất biến: khi chọn và cố định đỉnh chưa xử lý có dist nhỏ nhất, khoảng cách đó đã là tối ưu nếu trọng số không âm. Bởi vậy, trọng số âm bị từ chối trước khi chạy.

GraphLab dùng quét tuyến tính để chọn đỉnh nhỏ nhất, không dùng binary heap. Với danh sách kề, độ phức tạp lõi của lựa chọn tuyến tính là $O(V^2+E)$ thời gian và O(V) bộ nhớ thuật toán, chưa tính trace. Không được ghi $O((V+E) \log V)$ như độ phức tạp của bản cài đặt hiện tại; đây chỉ là cận thường gặp khi thay phần chọn đỉnh bằng hàng đợi ưu tiên binary heap.

3.5. A* — tìm kiếm có heuristic

A* ưu tiên đỉnh theo f(v)=g(v)+h(v), trong đó g(v) là chi phí thật từ nguồn tới v đã biết và h(v) ước lượng chi phí còn lại đến đích. Dùng g để cập nhật đường đi và previous; phần open chứa đỉnh còn chờ mở rộng, closed ghi các đỉnh đã mở rộng. Đỉnh được chọn có f nhỏ nhất.

Để tránh giả định rằng tọa độ vẽ tương đương trực tiếp với trọng số cạnh, GraphLab xây dựng heuristic Euclid co giãn. Đặt d_geo(u,v) là khoảng cách Euclid giữa hai tọa độ đầu mút. Chọn hệ số k = min(w(u,v)/d_geo(u,v)) trên các cạnh có d_geo > 0; nếu không có tỷ số hữu hạn thì lấy k=0. Sau đó h(n)=k*d_geo(n,t). Do $k*d_geo(u,v) \le w(u,v)$ trên mọi cạnh và khoảng cách Euclid thỏa bất đẳng thức tam giác, heuristic này nhất quán (consistent) với trọng số đã nhập. Nếu có cạnh trọng số 0 với hai tọa độ khác nhau, k trở thành 0 và A* tương đương Dijkstra về thứ tự ưu tiên; điều này làm heuristic yếu hơn nhưng vẫn an toàn.

Bản triển khai chọn đỉnh open có f nhỏ nhất bằng quét tuyến tính, nên lõi có cận xấu nhất $O(V^2+E)$. Hiệu quả A* phụ thuộc sức mạnh của heuristic; trong trường hợp heuristic bằng 0, thứ tự ưu tiên tương đương Dijkstra. Trọng số âm bị từ chối.

3.6. Khôi phục đường đi và chi phí của trace

Khi thuật toán kết thúc, bắt đầu từ goal, lần theo previous ngược về start rồi đảo ngược danh sách. Bộ kiểm tra vòng lặp phòng trường hợp dữ liệu cha bất thường. Nếu đích khác nguồn và không có chuỗi cha hợp lệ, đường đi rỗng. Với start == goal, kết quả là [start] và chi phí 0.

Ngoài chi phí của thuật toán lõi, GraphLab cần lưu lịch sử để hỗ trợ lùi bước và phát lại. Nếu có S snapshot và mỗi snapshot sao chép mảng/từ điển có tối đa O(V) phần tử thì chi phí bổ sung cho việc tạo/lưu trace có thể đạt $O(S \cdot V)$ thời gian và bộ nhớ; ở đây số thao tác trace phụ thuộc V+E, nên có thể mô tả xấp xỉ là $O(V(V+E))$ cho lớp trace ở trường hợp xấu. Giới hạn 15 đỉnh giữ trải nghiệm tương tác ổn định. Khi mở rộng đồ thị lớn, có thể dùng snapshot chênh lệch (delta), nhật ký thao tác hoặc chỉ lưu trạng thái thay đổi.

4. Ví dụ minh họa từng bước

Ví dụ dùng đúng đồ thị mẫu của ứng dụng, chọn Dijkstra, nguồn A, đích F. Các cạnh vô hướng và trọng số được liệt kê ở mục 2.5. Phần này phản ánh trace trả về bởi runAlgorithm(sampleGraph, "Dijkstra", "A", "F"); số bước đánh số 1–34 tương ứng với bộ đếm bước trong giao diện. ∞ nghĩa là chưa có đường nào được phát hiện tới đỉnh đó. Trong bảng, parent được ghi theo dạng đỉnh:cha; — nghĩa là chưa có cha.

Thuật toán lần lượt chọn A(0), C(2), B(3), D(8), E(10), F(13). Khi đi từ C sang B, khoảng cách giảm từ 4 xuống 3; khi đi từ B sang D, khoảng cách giảm từ 10 xuống 8; qua E, khoảng cách đến F giảm từ 14 xuống 13. Kết quả cuối là A → C → B → D → E → F, chi phí 13.

4.1. Nhật ký đầy đủ của 34 snapshot

Bảng 1 — Thao tác và cấu trúc chờ/đã thăm

Bước	Thao tác và diễn giải	Current	Frontier	Visited
1	Khởi tạo — Đặt khoảng cách từ A đến chính nó bằng 0.	A	A	∅
2	Chọn đỉnh A — Đỉnh chưa cố định có khoảng cách tạm thời nhỏ nhất là A (0).	A	∅	A
3	Nới lỏng cạnh A → B — So sánh 0 + 4 với khoảng cách hiện tại của B.	A	B, C, D, E, F	A
4	Cập nhật khoảng cách B — Cập nhật dist[B] = 4; cha của B là A.	A	B, C, D, E, F	A
5	Nới lỏng cạnh A → C — So sánh 0 + 2 với khoảng cách hiện tại của C.	A	B, C, D, E, F	A
6	Cập nhật khoảng cách C — Cập nhật dist[C] = 2; cha của C là A.	A	B, C, D, E, F	A
7	Chọn đỉnh C — Đỉnh chưa cố định có khoảng cách tạm thời nhỏ nhất là C (2).	C	B	A, C
8	Nới lỏng cạnh C → A — So sánh 2 + 2 với khoảng cách hiện tại của A.	C	B, D, E, F	A, C
9	Nới lỏng cạnh C → B — So sánh 2 + 1 với khoảng cách hiện tại của B.	C	B, D, E, F	A, C
10	Cập nhật khoảng cách B — Cập nhật dist[B] = 3; cha của B là C.	C	B, D, E, F	A, C
11	Nới lỏng cạnh C → D — So sánh 2 + 8 với khoảng cách hiện tại của D.	C	B, D, E, F	A, C
12	Cập nhật khoảng cách D — Cập nhật dist[D] = 10; cha của D là C.	C	B, D, E, F	A, C
13	Nới lỏng cạnh C → E — So sánh 2 + 10 với khoảng cách hiện tại của E.	C	B, D, E, F	A, C
14	Cập nhật khoảng cách E — Cập nhật dist[E] = 12; cha của E là C.	C	B, D, E, F	A, C
15	Chọn đỉnh B — Đỉnh chưa cố định có khoảng cách tạm thời nhỏ nhất là B (3).	B	D, E	A, C, B
16	Nới lỏng cạnh B → A — So sánh 3 + 4 với khoảng cách hiện tại của A.	B	D, E, F	A, C, B
17	Nới lỏng cạnh B → C — So sánh 3 + 1 với khoảng cách hiện tại của C.	B	D, E, F	A, C, B
18	Nới lỏng cạnh B → D — So sánh 3 + 5 với	B	D, E, F	A, C, B

Bước	Thao tác và diễn giải	Current	Frontier	Visited
	khoảng cách hiện tại của D.			
19	Cập nhật khoảng cách D — Cập nhật dist[D] = 8; cha của D là B.	B	D, E, F	A, C, B
20	Chọn đỉnh D — Đỉnh chưa cố định có khoảng cách tạm thời nhỏ nhất là D (8).	D	E	A, C, B, D
21	Nới lỏng cạnh D → B — So sánh 8 + 5 với khoảng cách hiện tại của B.	D	E, F	A, C, B, D
22	Nới lỏng cạnh D → C — So sánh 8 + 8 với khoảng cách hiện tại của C.	D	E, F	A, C, B, D
23	Nới lỏng cạnh D → E — So sánh 8 + 2 với khoảng cách hiện tại của E.	D	E, F	A, C, B, D
24	Cập nhật khoảng cách E — Cập nhật dist[E] = 10; cha của E là D.	D	E, F	A, C, B, D
25	Nới lỏng cạnh D → F — So sánh 8 + 6 với khoảng cách hiện tại của F.	D	E, F	A, C, B, D
26	Cập nhật khoảng cách F — Cập nhật dist[F] = 14; cha của F là D.	D	E, F	A, C, B, D
27	Chọn đỉnh E — Đỉnh chưa cố định có khoảng cách tạm thời nhỏ nhất là E (10).	E	F	A, C, B, D, E
28	Nới lỏng cạnh E → C — So sánh 10 + 10 với khoảng cách hiện tại của C.	E	F	A, C, B, D, E
29	Nới lỏng cạnh E → D — So sánh 10 + 2 với khoảng cách hiện tại của D.	E	F	A, C, B, D, E
30	Nới lỏng cạnh E → F — So sánh 10 + 3 với khoảng cách hiện tại của F.	E	F	A, C, B, D, E
31	Cập nhật khoảng cách F — Cập nhật dist[F] = 13; cha của F là E.	E	F	A, C, B, D, E
32	Chọn đỉnh F — Đỉnh chưa cố định có khoảng cách tạm thời nhỏ nhất là F (13).	F	∅	A, C, B, D, E, F
33	Đã cố định đỉnh đích — Khoảng cách ngắn nhất đến F là 13.	F	∅	A, C, B, D, E, F
34	Hoàn tất — Đường đi A → C → B → D → E → F có tổng trọng số 13.	F	∅	A, C, B, D, E, F

Bảng 2 — Toàn trạng thái khoảng cách và cha sau mỗi bước

Bước	dist(A,B,C,D,E,F)	parent(A,B,C,D,E,F)
1	A=0, B=∞, C=∞, D=∞, E=∞, F=∞	A=—, B=—, C=—, D=—, E=—, F=—
2	A=0, B=∞, C=∞, D=∞, E=∞, F=∞	A=—, B=—, C=—, D=—, E=—, F=—
3	A=0, B=∞, C=∞, D=∞, E=∞, F=∞	A=—, B=—, C=—, D=—, E=—, F=—
4	A=0, B=4, C=∞, D=∞, E=∞, F=∞	A=—, B=A, C=—, D=—, E=—, F=—
5	A=0, B=4, C=∞, D=∞, E=∞, F=∞	A=—, B=A, C=—, D=—, E=—, F=—
6	A=0, B=4, C=2, D=∞, E=∞, F=∞	A=—, B=A, C=A, D=—, E=—, F=—
7	A=0, B=4, C=2, D=∞, E=∞, F=∞	A=—, B=A, C=A, D=—, E=—, F=—
8	A=0, B=4, C=2, D=∞, E=∞, F=∞	A=—, B=A, C=A, D=—, E=—, F=—
9	A=0, B=4, C=2, D=∞, E=∞, F=∞	A=—, B=A, C=A, D=—, E=—, F=—
10	A=0, B=3, C=2, D=∞, E=∞, F=∞	A=—, B=C, C=A, D=—, E=—, F=—
11	A=0, B=3, C=2, D=∞, E=∞, F=∞	A=—, B=C, C=A, D=—, E=—, F=—

Bước	dist(A,B,C,D,E,F)	parent(A,B,C,D,E,F)
12	A=0, B=3, C=2, D=10, E=∞, F=∞	A=—, B=C, C=A, D=C, E=—, F=—
13	A=0, B=3, C=2, D=10, E=∞, F=∞	A=—, B=C, C=A, D=C, E=—, F=—
14	A=0, B=3, C=2, D=10, E=12, F=∞	A=—, B=C, C=A, D=C, E=C, F=—
15	A=0, B=3, C=2, D=10, E=12, F=∞	A=—, B=C, C=A, D=C, E=C, F=—
16	A=0, B=3, C=2, D=10, E=12, F=∞	A=—, B=C, C=A, D=C, E=C, F=—
17	A=0, B=3, C=2, D=10, E=12, F=∞	A=—, B=C, C=A, D=C, E=C, F=—
18	A=0, B=3, C=2, D=10, E=12, F=∞	A=—, B=C, C=A, D=C, E=C, F=—
19	A=0, B=3, C=2, D=8, E=12, F=∞	A=—, B=C, C=A, D=B, E=C, F=—
20	A=0, B=3, C=2, D=8, E=12, F=∞	A=—, B=C, C=A, D=B, E=C, F=—
21	A=0, B=3, C=2, D=8, E=12, F=∞	A=—, B=C, C=A, D=B, E=C, F=—
22	A=0, B=3, C=2, D=8, E=12, F=∞	A=—, B=C, C=A, D=B, E=C, F=—
23	A=0, B=3, C=2, D=8, E=12, F=∞	A=—, B=C, C=A, D=B, E=C, F=—
24	A=0, B=3, C=2, D=8, E=10, F=∞	A=—, B=C, C=A, D=B, E=D, F=—
25	A=0, B=3, C=2, D=8, E=10, F=∞	A=—, B=C, C=A, D=B, E=D, F=—
26	A=0, B=3, C=2, D=8, E=10, F=14	A=—, B=C, C=A, D=B, E=D, F=D
27	A=0, B=3, C=2, D=8, E=10, F=14	A=—, B=C, C=A, D=B, E=D, F=D
28	A=0, B=3, C=2, D=8, E=10, F=14	A=—, B=C, C=A, D=B, E=D, F=D
29	A=0, B=3, C=2, D=8, E=10, F=14	A=—, B=C, C=A, D=B, E=D, F=D
30	A=0, B=3, C=2, D=8, E=10, F=14	A=—, B=C, C=A, D=B, E=D, F=D
31	A=0, B=3, C=2, D=8, E=10, F=13	A=—, B=C, C=A, D=B, E=D, F=E
32	A=0, B=3, C=2, D=8, E=10, F=13	A=—, B=C, C=A, D=B, E=D, F=E
33	A=0, B=3, C=2, D=8, E=10, F=13	A=—, B=C, C=A, D=B, E=D, F=E
34	A=0, B=3, C=2, D=8, E=10, F=13	A=—, B=C, C=A, D=B, E=D, F=E

4.2. Diễn giải các điểm quyết định

- **Khởi tạo (bước 1):** dist[A]=0; các đỉnh còn lại có khoảng cách vô cùng và chưa có cha.
- **Mở rộng A (bước 2–6):** đặt dist[B]=4, dist[C]=2. Dijkstra chọn C trước B do 2 < 4.
- **Mở rộng C (bước 7–14):** cạnh C–B(1) cho ứng viên 2+1=3, tốt hơn 4, nên đổi parent[B]=C; các ứng viên tới D và E lần lượt là 10 và 12.
- **Mở rộng B (bước 15–19):** qua cạnh B–D(5), ứng viên mới là 3+5=8, tốt hơn 10 nên parent[D]=B.
- **Mở rộng D (bước 20–26):** cập nhật dist[E]=10 qua cạnh D–E(2) và lần đầu phát hiện dist[F]=14 qua D–F(6).
- **Mở rộng E (bước 27–31):** dist[E]=10, qua E–F(3) nhận 13, tốt hơn 14; đổi parent[F]=E.
- **Chọn F và kết thúc (bước 32–34):** F được chọn với chi phí 13. Truy vết F ← E ← D ← B ← C ← A rồi đảo ngược để được đường đi cuối.

Các trạng thái, parent và khoảng cách trong bảng lấy trực tiếp từ tệp testcases/sample-dijkstra-trace.json được sinh từ engine và được lưu trong gói nộp. Khi quay video, cần mở app React, chọn Dijkstra, đặt A→F, nhấn tiến từng bước và đối chiếu bộ đếm với bảng trên. Không dùng mô phỏng riêng của slide deck làm bằng chứng thay thế cho engine React.

5. Thiết kế giao diện và quá trình phát triển

5.1. Nguyên tắc thiết kế

Giao diện được thiết kế theo hướng "quan sát thuật toán trước, trang trí sau". Màn hình chính dùng nền tối, các vùng nội dung tách bằng viền nhẹ và bảng màu trạng thái có ý nghĩa nhất quán. Phần Graph Visualization biểu diễn đỉnh/cạnh bằng SVG; phần Execution Trace trình bày hành động hiện tại; vùng Frontier, Visited Nodes và Distance/G-F Score Table cho biết cấu trúc dữ liệu đang thay đổi; vùng Path Result chỉ công bố kết quả sau khi người dùng đi tới bước cuối của trace.

Mỗi màu không chỉ có tính trang trí: đỉnh đang xét, đỉnh đã thăm, frontier và đường đi cuối được biểu diễn thành các trạng thái riêng. Cạnh đang xét được làm nổi bật; nếu đồ thị có hướng, mũi tên thể hiện chiều cạnh. Cách biểu diễn này giúp người dùng nối trực tiếp giữa thao tác trong trace và thay đổi trên đồ thị.

5.2. Kiến trúc chức năng

```
src/main.tsx
└── App.tsx — trạng thái giao diện, input, điều khiển và kết quả
    ├── graph.ts — mô hình dữ liệu, thuật toán, validation và trace
    ├── SVG — đỉnh, cạnh, trọng số, hướng, trạng thái hiện tại
    └── styles.css — bố cục, màu trạng thái, responsive UI

presentation.html — bài thuyết trình tương tác độc lập, bổ trợ cho app
testcases/ — JSON fixtures, test runner, kết quả test và trace mẫu
docs/ — report nguồn, Word, PDF
video-demo/ — kịch bản/checklist ghi hình
```

graph.ts không phụ thuộc React, do vậy có thể kiểm thử thuật toán riêng. App.tsx gọi runAlgorithm khi graph/algorithm/start/goal thay đổi, rồi render trạng thái ở stepIndex. reset, next, prev và playback điều khiển chỉ số bước; trace được tính lại khi dữ liệu thay đổi. Tách thuật toán khỏi giao diện cho phép kiểm thử logic mà không phụ thuộc animation hoặc trình duyệt.

5.3. Quá trình thiết kế và hoàn thiện chức năng

Giai đoạn 1 — Chuyển yêu cầu thành mô hình dữ liệu

Yêu cầu của bài tập được chia thành ba luồng: (1) nhập dữ liệu, (2) quan sát từng bước, (3) xem output. Trước tiên xác định GraphNode, GraphEdge, GraphData, Step, RunResult. Một quyết định có ý nghĩa là không chỉ trả về đường đi cuối cùng; engine phải trả steps[] để giao diện có thể lùi và tiến mà không tự suy luận lại trạng thái. Mỗi snapshot sao chép các mảng và bản đồ khoảng cách/cha để trạng thái bước trước không bị thay đổi ngoài ý muốn khi thuật toán tiếp tục chạy.

Giai đoạn 2 — Xây dựng Algorithm Engine độc lập

BFS, DFS và Dijkstra được xây dựng trên chung một API runAlgorithm(graph, algorithm, start, goal). Danh sách kề được tạo từ danh sách cạnh; cờ directed quyết định có thêm cung ngược hay không. Engine kiểm tra nguồn/đích, mã đỉnh, đầu mút cạnh, trọng số hữu hạn và tiền điều kiện trọng số không âm của Dijkstra/A*. Sau mỗi thao tác quan trọng, engine lưu một snapshot gồm tiêu đề và giải thích tiếng Việt, current, frontier, visited, distances, previous, activeEdge, số lần kiểm tra cạnh và trạng thái.

Sau khi kiểm thử các thuật toán cơ bản, A* được bổ sung vào cùng API. Thách thức là tọa độ đỉnh trên SVG chỉ là vị trí hiển thị, không mặc nhiên là đơn vị khoảng cách tương thích với trọng số. Nếu dùng trực tiếp khoảng cách Euclid, heuristic có thể đánh giá quá cao chi phí còn lại và làm mất bảo đảm tối ưu. Cách giải quyết là tìm hệ số co giãn nhỏ nhất $k=\min(w/d_{geo})$ trên các cạnh có độ dài hình học dương rồi dùng $h(n)=k*d_{geo}(n,t)$. Cách xây dựng này bảo đảm heuristic không vượt quá trọng số cạnh theo bất đẳng thức tam giác; các bài test ngẫu nhiên tiếp tục đối chiếu A* với oracle Bellman–Ford.

Giai đoạn 3 — Thiết kế điều khiển Input

Panel bên trái nhóm chức năng theo thuật toán, thiết lập đồ thị và hành động nhanh. Người dùng chọn thuật toán, start, goal, bật/tắt đồ thị có hướng, thêm đỉnh, thêm cạnh, xóa cạnh cuối hoặc khôi phục dữ liệu mẫu. Vì Dijkstra/A* không hợp lệ với trọng số âm, control nhập trọng số có min=0 khi dùng hai thuật toán này và hàm xử lý thêm cạnh hiển thị thông báo lỗi nếu giá trị không hữu hạn, đầu mút trùng nhau hoặc trọng số âm. Validation cũng tồn tại trong engine để bảo vệ trường hợp gọi trực tiếp ngoài giao diện.

Giai đoạn 4 — Đồng bộ trạng thái thuật toán với SVG và trace

Mỗi lần render, giao diện tạo tập visited và frontier từ snapshot hiện tại. Đỉnh current được đánh dấu khác với đỉnh đã thăm; cạnh activeEdge được nhấn mạnh. Đường đi cuối không hiển thị ngay từ đầu, tránh làm lộ đáp án trước khi người học lần theo quá trình. Bảng khoảng cách hiển thị dist[v] với BFS/DFS/Dijkstra; với A* hiển thị cả g[v] và f[v] để giải thích thứ tự ưu tiên. Khu vực trace dùng số bước và thanh tiến độ; người dùng có thể lùi, tiến, reset hoặc tự phát lại theo thời gian.

Một vấn đề thường gặp trong demo trực quan là kết quả cuối có thể bị hiển thị ngay cả khi người dùng đang ở bước đầu, vì engine đã tính trước toàn bộ trace. Điều này làm yếu giá trị sư phạm. Vì vậy giao diện chỉ hiển thị path/cost/metrics cuối khi stepIndex đã tới snapshot cuối cùng; trước đó phần kết quả thể hiện rằng thuật toán vẫn đang chạy.

Giai đoạn 5 — Kiểm thử và đồng bộ tài liệu

Bộ testcase được tổ chức thành dữ liệu JSON để có thể lặp lại. Runner kiểm tra đường đi, chi phí, tính hợp lệ các cạnh trên đường, tiền điều kiện, các trường trace và đối chiếu chi phí Dijkstra/A* với thuật toán Bellman–Ford viết độc lập cho kiểm thử. Ngoài 26 fixture cố định, 100 đồ thị được sinh bằng bộ sinh giả ngẫu nhiên có seed cố định; mỗi đồ thị được kiểm tra bằng cả Dijkstra và A*, tạo 200 phép so sánh chi phí tái hiện được.

Trong quá trình rà soát, phát hiện sự không thống nhất giữa các phần: phiên bản app ban đầu chỉ có BFS/DFS/Dijkstra dù phần trình bày đã nhắc A*; report còn để chỗ trống cho kết quả thực nghiệm và chưa có bảng trace được sinh từ engine; path cuối xuất hiện trong UI ngay cả khi chưa tới bước cuối. Các điểm này được xử lý bằng cách bổ sung A* vào app, thêm testcase và log trace, sửa điều kiện hiển thị output và viết lại báo cáo. Bài thuyết trình HTML vẫn là công cụ trình bày độc lập; bản của gói này dùng cùng đồ thị mẫu A–F nhưng không thay thế việc quay video trên app React.

5.4. Công nghệ và lý do lựa chọn

Công nghệ	Vai trò	Lý do
React	UI theo component	Quản lý các vùng tương tác và render lại theo trạng thái
TypeScript	Kiểu dữ liệu	Làm rõ hợp đồng giữa Graph Model, Engine và UI
Vite	Dev server/build	Quy trình chạy và đóng gói frontend gọn
SVG	Đồ thị	Đỉnh/cạnh/trọng số sắc nét, trạng thái tô màu được cập nhật theo bước
Lucide React	Icon	Bộ biểu tượng thống nhất cho nút và panel
Node.js test runner + TypeScript compiler	Kiểm thử engine	Kiểm tra logic độc lập, không cần browser để thực thi engine

6. Thiết kế thực nghiệm và bộ testcase

6.1. Chiến lược thiết kế testcase

Bộ test được chia theo các lớp sau:

- **Thông thường:** đường đi tồn tại, nhiều nhánh, đồ thị có trọng số và đường tối ưu không phải cạnh trực tiếp.
- **Biên:** chỉ một đỉnh, nguồn trùng đích, không có cạnh, trọng số bằng 0.
- **Đồ thị có hướng/vô hướng:** kiểm tra cung ngược không được sử dụng trong đồ thị có hướng.
- **Không liên thông:** đích cô lập hoặc nằm ở thành phần không thể tới.
- **Nhiều nghiệm:** hai đường có cùng chi phí tối ưu; chỉ so sánh một chuỗi cụ thể nếu quy tắc phá hòa được xác định, nếu không thì so sánh cost/path validity.
- **Không hợp lệ:** trọng số âm với Dijkstra/A*, cạnh tham chiếu đỉnh không tồn tại, tập đỉnh rỗng hoặc mã đỉnh trùng.
- **Đồ thị lớn hơn:** nhiều đỉnh và nhánh hơn trường hợp tối thiểu.
- **Đối chiếu oracle:** trên đồ thị giả ngẫu nhiên không âm, đối chiếu chi phí Dijkstra và A* với Bellman–Ford độc lập.

6.2. Điều kiện Pass/Fail

Một testcase đạt khi các điều kiện được định nghĩa cho testcase đều đúng. Với đường đi có kết quả, kiểm tra nguồn, đích, mọi cặp đỉnh liên tiếp có cạnh hợp lệ, tổng chi phí tính lại từ cạnh và totalCost của engine khớp nhau. Dijkstra/A* được so sánh với Bellman–Ford để xác minh chi phí tối ưu; BFS được kiểm tra số cạnh không dài hơn đường BFS tham chiếu; DFS chỉ cần đường hợp lệ và đúng hành vi tìm kiếm. Với input không hợp lệ, hệ thống phải trả về path rỗng, totalCost=null và có trạng thái/tiêu đề lỗi thích hợp.

6.3. Danh mục 26 fixtures cố định

ID	Nhóm	Thuật toán	Tiêu chí chính
TC01	Thường	BFS	Đường ít cạnh; phá hòa theo thứ tự kề
TC02	Thường	DFS	Tìm đích qua một nhánh
TC03	Có trọng số	Dijkstra	Chi phí tối ưu bằng 6
TC04	Rời rạc	Dijkstra	Không có đường tới đích
TC05	Biên	BFS	Một đỉnh, nguồn trùng đích, chi phí 0
TC06	Biên	Dijkstra	Cạnh trọng số 0
TC07	Không hợp lệ	Dijkstra	Từ chối trọng số âm
TC08	Có hướng	BFS	Không đi ngược chiều cạnh
TC09	Có hướng	Dijkstra	Relax cạnh, chi phí tối ưu 5
TC10	Biên	DFS	Đỉnh đích cô lập
TC11	Nhiều nghiệm	Dijkstra	Hai đường tối ưu cùng chi phí 3
TC12	Lớn hơn	BFS	Tám đỉnh, nhiều nhánh
TC13	Thường	A*	Chi phí trùng Dijkstra, bằng 6
TC14	Biên	A*	Nguồn trùng đích
TC15	Rời rạc	A*	Đích không thể tới
TC16	Không hợp lệ	A*	Từ chối trọng số âm
TC17	Số thực	Dijkstra	Chi phí thập phân 1.75
TC18	Không hợp lệ	BFS	Tập đỉnh rỗng
TC19	Không hợp lệ	Dijkstra	Cạnh có đầu mút không tồn tại

ID	Nhóm	Thuật toán	Tiêu chí chính
TC20	Không hợp lệ	DFS	Mã đỉnh trùng
TC21	Có hướng	A*	Đường theo cung, chi phí 6
TC22	Chu trình	Dijkstra	Có chu trình nhưng vẫn tối ưu chi phí 4
TC23	Có hướng	BFS	Đường có hướng tồn tại
TC24	Cạnh song song	Dijkstra	Chọn trọng số nhỏ hơn, chi phí 2
TC25	Số 0	A*	Cạnh 0, heuristic an toàn, chi phí 1
TC26	Có hướng	DFS	Minh họa thứ tự LIFO

Input đầy đủ từng fixture được lưu trong testcases/testcases.json, không viết lại toàn bộ danh sách cạnh trong report để tránh lặp dữ liệu. Runner testcases/run-tests.mjs sử dụng đúng JSON này. Kết quả chi tiết sau lần chạy cuối nằm ở testcases/last-run-results.json.

7. Kết quả kiểm thử và đánh giá

7.1. Kết quả thực nghiệm thuật toán

Lần chạy engine gần nhất trong môi trường đóng gói cho kết quả:

Hạng mục	Kết quả
Fixture cố định	26
Fixture đạt	26
Fixture không đạt	0
Đồ thị giả ngẫu nhiên có seed cố định	100
So sánh chi phí Dijkstra/A* với Bellman–Ford	200/200 đạt
Đối chiếu chi phí trên đồ thị mẫu	Dijkstra = A* = 13

Các số trên là kết quả thật từ runner của engine tại thời điểm tạo gói; chúng không đại diện cho kiểm thử end-to-end của trình duyệt. Runner cũng kiểm tra tính hợp lệ của path, các trường snapshot, hành vi với input không hợp lệ và tính nhất quán giữa chi phí engine với tổng trọng số tính lại trên đường đi.

7.2. Các tiêu chí đánh giá

- **Tính đúng đắn thuật toán:** được kiểm chứng bằng 26 fixture và đối chiếu oracle trên 100 đồ thị giả ngẫu nhiên không âm cho Dijkstra/A*. BFS có kiểm tra đường ít cạnh; DFS có kiểm tra tính hợp lệ và hành vi tìm kiếm.
- **Khả năng quan sát:** engine ghi lại current, frontier, visited, khoảng cách, quan hệ cha và cạnh đang xét; A* bổ sung g/h/f.
- **Khả năng tương tác:** giao diện có bộ chọn thuật toán, nguồn/đích, hướng đồ thị, thêm đỉnh/cạnh, xóa cạnh cuối, khôi phục mẫu, tiến/lùi, tự chạy và reset.
- **Tính sáng tạo:** cùng đồ thị có thể đối chiếu bốn chiến lược; A* dùng heuristic được xây dựng an toàn thay vì coi tọa độ vẽ là khoảng cách thực; có trình chiếu web với điều hướng bàn phím và hoạt ảnh.
- **Hiệu năng:** bài tập tập trung vào tính đúng và khả năng quan sát với đồ thị nhỏ (tối đa 15 đỉnh được thêm qua UI); chưa có benchmark đáng tin cậy cho thời gian trình duyệt trên kích thước lớn, vì vậy không báo cáo số liệu hiệu năng giả định.

8. Hạn chế và hướng phát triển

Hạn chế hiện tại gồm giới hạn đồ thị nhỏ, vị trí đỉnh không kéo-thả, chưa nhập đồ thị từ file, chưa có so sánh nhiều thuật toán đồng thời trên cùng màn hình, chưa đo benchmark trình duyệt, và heuristic A* chủ động chọn yếu để đảm bảo nhất quán với mọi trọng số cạnh hợp lệ. Tính yếu của heuristic làm A* vẫn có thể khám phá nhiều đỉnh, nhất là khi có cạnh trọng số 0.

Hướng phát triển:

1. Thêm chế độ sinh đồ thị với seed, đồ thị lưới và heuristic Manhattan/Euclid theo mô hình bản đồ có đơn vị rõ ràng.
2. Bổ sung cấu trúc binary heap cho Dijkstra/A* để so sánh cận O((V+E) log V) với bản quét tuyến tính hiện tại.
3. Bổ sung bảng so sánh song song cho số đỉnh mở rộng, số cạnh kiểm tra, độ dài đường đi và thời gian thực đo.
4. Thêm kéo-thả, nhập/xuất JSON/CSV, preset testcase và chế độ hiển thị previous/dist/g,h,f trực quan hơn.
5. Tối ưu trace bằng delta snapshots khi làm việc với đồ thị lớn.

9. Tài liệu tham khảo

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. Các chương về graph algorithms, BFS, DFS và shortest paths.
2. Sedgewick, R., Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
3. Hart, P. E., Nilsson, N. J., Raphael, B. (1968). "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136
4. React Documentation. https://react.dev/ (truy cập 18/10/2026).
5. Vite Documentation. https://vite.dev/ (truy cập 18/10/2026).
6. TypeScript Documentation. https://www.typescriptlang.org/docs/ (truy cập 18/10/2026).
7. Node.js Documentation — CLI and built-in runtime. https://nodejs.org/docs/ (truy cập 18/10/2026).

Phụ lục A. Cách chạy và cấu trúc gói nộp

Yêu cầu tiên quyết: Cần cài đặt môi trường hỗ trợ nodejs.

```
npm install
npm run dev
# Mở URL do Vite in ra, thường là http://localhost:5173

# Kiểm tra type/build production
npm run build

# Chạy bộ kiểm thử engine: 26 fixtures + 200 đối chiếu đồ thị ngẫu nhiên
npm run test:engine
```

Cấu trúc ZIP được tổ chức thành src/ (mã nguồn React/TypeScript), testcases/ (fixtures, runner, log và trace mẫu), docs/ (report Word/PDF), video-demo, presentation.html (slide web tương tác), cùng cấu hình Vite/TypeScript và hướng dẫn tại README.md. Bài thuyết trình HTML có thể mở trực tiếp bằng trình duyệt; ứng dụng React phải chạy qua Vite. Hai phần phục vụ hai mục đích khác nhau và không được nhầm lẫn bài thuyết trình HTML với video demo đã quay.

Phụ lục B. Nhật ký sử dụng AI và cách kiểm chứng

AI được dùng như công cụ hỗ trợ phân tích yêu cầu, đề xuất cấu trúc, phác thảo/hiệu chỉnh mã, xây dựng testcase. AI không thay thế bước chạy chương trình hoặc xác minh kết quả thực nghiệm và viết báo cáo. Mục tiêu là để tối ưu thời gian thực hiện bài tập lớn trong 1 ngày tính từ lúc nhận thông tin về bài tập. Các mô tả dưới đây ghi lại nhóm yêu cầu đã đưa ra trong hội thoại theo dạng tóm lược, không giả định đây là bản xuất nguyên văn của toàn bộ prompt từng lần.

Công đoạn	Prompt/brief đã sử dụng (tóm lược)	Kết quả hỗ trợ	Vấn đề cần kiểm chứng	Cách kiểm chứng/chỉnh sửa
Chọn đề tài và stack	Chọn nhóm thuật toán đồ thị BFS/DFS/Dijkstra/A*, cân bằng độ khó, UI và giá trị học thuật; dùng React + TypeScript + Vite	Định hướng GraphLab với engine tách khỏi UI	Đề tài phải thống nhất từ model đến demo/report	Đối chiếu trực tiếp với đề bài; định nghĩa Input/Output/constraints
Tạo demo	Xây dựng giao diện nhập dữ liệu, thể hiện trace từng bước, kết quả và điều khiển	React SVG playground, step trace, thêm/xóa cạnh, hướng đồ thị, playback	Kết quả có thể hiển thị sớm hơn bước cuối; validation có thể im lặng	Chỉ hiện output cuối ở bước cuối; thêm thông báo input lỗi; test engine riêng
Mở rộng A*	Bổ sung A* và làm rõ heuristic, không dựa vào giả định tọa độ bản vẽ là đơn vị trọng số	g, h, f, frontier và parent trong trace	Heuristic tùy ý có thể không admissible/consistent	Dùng hệ số co giãn nhỏ nhất từ các cạnh; đối chiếu 100 graph × 2 thuật toán với Bellman–Ford
Bộ testcase	Tạo nhóm testcase thường, biên, dữ liệu sai, có hướng, nhiều đường, trọng số 0 và đồ thị lớn hơn	26 fixture JSON + runner và báo cáo kết quả	Một output path cụ thể không phải đáp án duy nhất khi có nhiều đường tối ưu	Dùng acceptable paths hoặc so sánh chi phí/đường hợp lệ; kiểm tra lại tổng trọng số
Trình bày	Tạo bài thuyết trình web có smooth scroll, điều khiển bàn phím, animation và mô phỏng	presentation.html gồm 8 slide và demo minh họa	Mô phỏng slide độc lập có thể lệch app chính	Đồng bộ đồ thị mẫu A–F; vẫn ghi rõ app React là demo chính